

Mars Rover Exploration Network

Baris Giorgio

Fionda Andrea
Petrilli Davide

Massaro Federico

Università degli Studi di Cassino e del Lazio Meridionale
Corso di Laurea Magistrale LM-32

A.A. 2025/2026

Indice

1	Introduzione e specifiche del problema	1
1.1	Introduzione	1
1.2	Specifiche del problema	2
1.3	Scenari delle missioni	2
1.3.1	Scenario 1 – Scientifico	3
1.3.2	Scenario 2 – Rotta solare	3
1.3.3	Scenario 3 – Relay di comunicazione	3
2	Algoritmi utilizzati	3
2.1	A*	3
2.1.1	Euristica A*: scenario scientifico	4
2.1.2	Euristica A*: scenario rotta solare	4
2.1.3	Euristica A*: scenario relay di comunicazione	5
2.2	BFS	5
3	Confronto tra A* e BFS	5
3.1	Differenze degli algoritmi	6
3.2	Differenze nei tre scenari	6
3.2.1	Scenario scientifico (Base Camp → Mineral Site).	6
3.2.2	Scenario solare (Water Cave → Observatory).	6
3.2.3	Scenario di comunicazione (Deep Crater → South Base).	6
3.3	Un'osservazione generale	7
4	Conclusioni	7
5	Immagini	7

1 Introduzione e specifiche del problema

1.1 Introduzione

La "Space Exploration Agency (SEA)" ha dispiegato una rete di 20 stazioni di esplorazione sulla superficie di Marte. Un rover autonomo "Pathfinder-X" deve navigare tra queste stazioni per

completare diverse missioni scientifiche, ottimizzando il percorso in base a vincoli specifici per ogni tipo di missione. Il rover opera in una regione marziara caratterizzata da:

- **Crateri vulcanici:** zone di difficile attraversamento
- **Pianure sabbiose:** zone con alte velocità ma ad alto rischio tempeste
- **Canyon rocciosi:** zone che offrono protezione ma sono caratterizzate da percorsi tortuosi
- **Altipiani:** zone con buona visibilità ma con alta esposizione alle radiazioni

1.2 Specifiche del problema

Viene scelta una topologia di rete esagonale che rappresenta la disposizione naturale delle stazioni sulla superficie marziana, con connessioni che riflettono la vicinanza geografica. All'interno di questa viene considerata una rete di 20 nodi (stazioni di esplorazione), ciascuno con caratteristiche specifiche:

- **Tipo di terreno:** cratere vulcanico, pianura sabbiosa, canyon roccioso o altipiano
- **Radiazione cosmica:** un valore tra 0 e 1 che indica il livello di radiazione al nodo i
- **Copertura solare:** un valore tra 0 e 1 che indica la quantità di luce solare disponibile al nodo i
- **Visibilità dell'orbiter:** un booleano che indica se il nodo i ha visibilità diretta verso l'orbiter in orbita marziana

Tabella 1: Caratteristiche dei nodi della rete marziana.

ID	Nome	Terreno	Radiazione	Solare	Visibilità
0	Base Camp	plateau	0.20	0.90	✓
1	Research Lab	plateau	0.30	0.85	✓
2	Crater Edge	crater	0.70	0.40	×
3	Deep Crater	crater	0.90	0.20	×
4	North Peak	plateau	0.50	0.95	✓
5	Observatory	plateau	0.40	1.00	✓
6	Canyon Entry	canyon	0.30	0.50	×
7	East Ridge	plateau	0.60	0.80	✓
8	Dust Plains	plain	0.40	0.70	✓
9	Storm Zone	plain	0.50	0.30	×
10	Ice Deposit	plain	0.30	0.60	✓
11	South Base	plateau	0.20	0.85	✓
12	Ancient River	canyon	0.40	0.45	×
13	Mineral Site	canyon	0.50	0.50	×
14	Volcanic Vent	crater	0.80	0.30	×
15	Central Hub	plateau	0.25	0.90	✓
16	Water Cave	canyon	0.35	0.10	×
17	West Outpost	plain	0.45	0.75	✓
18	Shelter Bay	canyon	0.20	0.40	×
19	Crossroads	plain	0.30	0.80	✓

Il costo di attraversamento tra due nodi dipende dalla distanza tra essi e dal tipo di terreno.

1.3 Scenari delle missioni

In questo progetto, il rover deve completare tre missioni distinte, ognuna con obiettivi e vincoli specifici. In questo paragrafo andremo a descrivere i tre scenari, evidenziando le differenze chiave e i requisiti di ottimizzazione per ciascuno.

1.3.1 Scenario 1 – Scientifico

Il rover deve partire dal Base Camp (nodo 0) e raggiungere il Mineral Site (nodo 13) minimizzando l'esposizione a radiazioni cosmiche. Il vincolo operativo è che nessun nodo con radiazione superiore a 0.6 può essere attraversato. La funzione di costo per questo scenario è definita come:

$$C_{\text{rad}}(u, v) = d(u, v) \cdot (1 + r_v)$$

dove $d(u, v)$ è la distanza tra i nodi u e v , e r_v è il livello di radiazione al nodo v . L'obiettivo è trovare un percorso che minimizzi la somma dei costi di attraversamento, garantendo che tutti i nodi del percorso abbiano $r_i \leq 0.6$.

1.3.2 Scenario 2 – Rotta solare

Il rover parte dalla Water Cave (nodo 16) con la batteria quasi scarica e deve raggiungere l'Observatory (nodo 5), che è una stazione di ricarica. L'obiettivo è massimizzare la copertura solare lungo il percorso, con il vincolo che ogni nodo intermedio deve avere copertura solare di almeno 0.5. La funzione di costo per questo scenario è definita come:

$$C_{\text{sol}}(u, v) = d(u, v) \cdot (2.0 - s_v)$$

dove s_v è la copertura solare al nodo v . In questo caso, i nodi con alta copertura solare ($s_v \rightarrow 1$) risultano più economici da attraversare, mentre quelli con bassa copertura ($s_v \rightarrow 0$) sono penalizzati. L'obiettivo è trovare un percorso che minimizzi la somma dei costi di attraversamento, garantendo che tutti i nodi del percorso abbiano $s_i \geq 0.5$.

1.3.3 Scenario 3 – Relay di comunicazione

Il rover è bloccato nel Deep Crater (nodo 3) senza segnale e deve raggiungere la South Base (nodo 11). Il vincolo operativo è che non si possono attraversare più di 2 nodi consecutivi senza visibilità dell'orbiter. La funzione di costo per questo scenario è definita come:

$$C_{\text{com}}(u, v) = d(u, v) \cdot \begin{cases} 0.8 & \text{se il nodo } v \text{ ha visibilità orbiter} \\ 1.5 & \text{altrimenti} \end{cases}$$

In questo caso, i nodi con visibilità dell'orbiter sono più economici da attraversare, mentre quelli senza visibilità sono penalizzati. L'obiettivo è trovare un percorso che minimizzi la somma dei costi di attraversamento, garantendo che non ci siano più di 2 nodi consecutivi senza visibilità dell'orbiter.

2 Algoritmi utilizzati

2.1 A*

A* è la scelta naturale quando si vuole guidare la ricerca verso il goal in modo intelligente. L'idea centrale è quella di valutare ogni nodo non solo in base a quanto è costato raggiungerlo, ma anche a quanto si stima costerà arrivare al goal da lì:

$$f(n) = g(n) + h(n)$$

dove $g(n)$ è il costo accumulato dal nodo di partenza e $h(n)$ è la stima euristica del costo residuo. Questa combinazione permette ad A* di esplorare preferenzialmente i nodi che sembrano promettenti, evitando di espandere zone del grafo lontane o costose senza motivo.

Nell'implementazione si usa una `PriorityQueue` (min-heap) per estrarre sempre il nodo con f minima, affiancata da un insieme `open_set_hash` per verificare in tempo costante se un nodo è già in coda. Ogni volta che si trova un percorso più economico verso un nodo già visitato, i punteggi vengono aggiornati e il nodo viene reinserito nella coda.

```

1 while not open_set.empty():
2     _, current = open_set.get()
3     if current == goal:
4         return reconstruct_path(came_from, current)
5     for neighbor in graph.neighbors(current):
6         tentative_g = g[current] + edge_cost(current, neighbor)
7         if tentative_g < g.get(neighbor, inf):
8             came_from[neighbor] = current
9             g[neighbor] = tentative_g
10            f[neighbor] = tentative_g + heuristic(neighbor, goal)
11            open_set.put((f[neighbor], neighbor))

```

Listing 1: Nucleo del loop A*

Una proprietà importante: se l'euristica è *ammissibile* (non sovrastima mai il costo reale) e *consistente* (soddisfa la disuguaglianza triangolare), A* garantisce di trovare la soluzione ottimale. Possiamo dire che le funzioni euristiche costituiscono l'intelligenza di A*. Ciascuno scenario ha una euristica diversa, costruita per riflettere il criterio di ottimizzazione della missione e per rispettare le proprietà (ammissibilità e consistenza) citate precedentemente.

2.1.1 Euristica A*: scenario scientifico

Il rover deve andare dal Base Camp (nodo 0) al Mineral Site (nodo 13) minimizzando l'esposizione a radiazioni cosmiche. Nessun nodo con radiazione superiore a 0.6 può far parte del percorso.

L'euristica è:

$$h_{\text{rad}}(n) = d(n, \text{goal}) \cdot (1 + r_n)$$

La motivazione della scelta è la seguente: la distanza euclidea da sola sottostima sempre il costo reale (è ammissibile per costruzione), ma non scoraggerebbe i nodi pericolosi. Moltiplicarla per $(1 + r_n)$ fa sì che i nodi ad alta radiazione risultino “più lontani” del necessario, spingendo A* a cercare percorsi alternativi. Poiché $r_n \in [0, 1]$, il moltiplicatore non scende mai sotto 1, mantenendo l'ammissibilità.

2.1.2 Euristica A*: scenario rotta solare

Il rover parte dalla Water Cave (nodo 16) con la batteria quasi scarica e deve raggiungere l'Observatory (nodo 5), che è una stazione di ricarica. Lungo il percorso, ogni nodo intermedio deve avere copertura solare di almeno 0.5.

$$h_{\text{sol}}(n) = d(n, \text{goal}) \cdot (2,0 - s_n)$$

Qui la logica è speculare: un nodo con alta copertura solare ($s_n \rightarrow 1$) ha un moltiplicatore che si avvicina a 1 – viene visto come “vicino” e quindi preferibile. Un nodo in ombra ($s_n \rightarrow 0$) ha moltiplicatore 2, risultando penalizzato. Il fattore 2.0 è scelto per garantire che il moltiplicatore sia sempre positivo e che la funzione rimanga ammissibile anche per $s_n = 0$.

2.1.3 Euristiche A*: scenario relay di comunicazione

Il rover è bloccato nel Deep Crater (nodo 3) senza segnale e deve raggiungere la South Base (nodo 11). Il vincolo operativo è che non si possono attraversare più di 2 nodi consecutivi senza visibilità dell'orbiter.

$$h_{\text{com}}(n) = d(n, \text{goal}) \cdot \begin{cases} 0,8 & \text{se il nodo ha visibilità orbiter} \\ 1,5 & \text{altrimenti} \end{cases}$$

I nodi con segnale vengono resi più “attraenti” riducendo la loro stima di costo; i nodi ciechi vengono penalizzati. Il rispetto del vincolo dei 2 hop consecutivi viene verificato esplicitamente dalla funzione `validate_communication_path` dopo la ricerca, che scarta il percorso se il vincolo è violato.

2.2 BFS

BFS esplora il grafo a strati concentrici partendo dal nodo di partenza, visitando prima tutti i vicini diretti, poi i vicini dei vicini, e così via. Questo approccio garantisce di trovare il percorso con il minor numero di archi (hop count), ma non tiene conto dei pesi né degli attributi dei nodi.

```

1 queue = deque([start])
2 visited = {start}
3 while queue:
4     current = queue.popleft()
5     for neighbor in graph.neighbors(current):
6         if neighbor not in visited:
7             visited.add(neighbor)
8             came_from[neighbor] = current
9             if neighbor == goal:
10                 return reconstruct_path(came_from, goal)
11             queue.append(neighbor)

```

Listing 2: Nucleo del loop BFS

BFS ha complessità $\mathcal{O}(V + E)$ garantita, indipendente da qualsiasi euristica. È uno strumento più “semplice” rispetto ad A*, ma utile come baseline e in contesti dove la semplicità del percorso (in termini di passi) è davvero ciò che conta.

Il limite principale in questo progetto è che BFS non può integrare direttamente i vincoli ambientali nella ricerca: li ignora durante l'esplorazione, e la validazione avviene solo a posteriori. Questo significa che in alcuni scenari BFS può trovare percorsi validi, ma in altri restituisce soluzioni che non superano i controlli.

3 Confronto tra A* e BFS

In questo paragrafo ci concentriamo sulle differenze sostanziali tra A* e BFS, evidenziando come queste si manifestano nei tre scenari di missione. Il confronto si basa su diversi aspetti chiave: ottimalità, capacità di integrare vincoli, complessità computazionale e comportamento atteso nei diversi contesti.

3.1 Differenze degli algoritmi

	A*	BFS
Ottimalità	✓	✓ (con archi con costo unitario)
Completezza	✓	✓
Percorso minimo (nodi espansi)	×	✓
Integra vincoli nella ricerca	✓	×
Complessità temporale	$\mathcal{O}(V + E)$	$\mathcal{O}(V + E)$

Tabella 2: Confronto sintetico tra i due algoritmi.

3.2 Differenze nei tre scenari

3.2.1 Scenario scientifico (Base Camp → Mineral Site).

Tabella 3: Confronto A* vs BFS – Scenario scientifico.

Metrica	A*	BFS
Criterio di esplorazione	$f(n) = g(n) + h_{\text{rad}}(n)$	hop count
Vincolo $r_i \leq 0,6$	integrato nell'euristica	ignorato
Ottimalità sul costo pesato	✓	×
Rischio di invalidazione	basso	alto
Nodi espansi (atteso)	$\ll V$	$\sim V$

A* integra r_i direttamente nella funzione di costo tramite il moltiplicatore $(1 + r_i)$, penalizzando i nodi ad alto rischio già durante l'esplorazione e orientando la ricerca verso regioni a bassa radiazione. BFS tratta tutti i nodi come equivalenti e può restituire un percorso che minimizza gli hop ma viola il vincolo $r_i \leq 0,6$ su uno o più nodi intermedi, risultando non valido dopo la fase di validazione. È lo scenario in cui il vantaggio qualitativo di A* è più netto.

3.2.2 Scenario solare (Water Cave → Observatory).

Tabella 4: Confronto A* vs BFS – Scenario solare.

Metrica	A*	BFS
Criterio di esplorazione	$f(n) = g(n) + h_{\text{sol}}(n)$	hop count
Vincolo $s_i \geq 0,5$	integrato nell'euristica	ignorato
Preferenza nodi soleggiati	✓	×
Rischio di invalidazione	basso	medio
Nodi espansi (atteso)	$\ll V$	$\sim V$

Il meccanismo è duale rispetto allo scenario precedente: il fattore $(2,0 - s_i)$ abbassa il costo stimato dei nodi con alta copertura solare, rendendoli preferibili durante l'esplorazione. BFS ignora s_i e può produrre percorsi che attraversano nodi con $s_i < 0,5$, fallendo la validazione sugli intermediari. Va notato che, su questa rete di 20 nodi, la densità degli archi e la distribuzione dei valori solari potrebbero comunque portare BFS a trovare una soluzione valida: in reti piccole il risultato dipende fortemente dalla topologia specifica e non è garantito a priori.

3.2.3 Scenario di comunicazione (Deep Crater → South Base).

Questo è lo scenario in cui il confronto è più sfumato. BFS minimizza per costruzione il numero di hop, il che implica statisticamente meno nodi ciechi attraversati; tuttavia non offre nessuna garanzia sul vincolo dei 2 hop consecutivi senza visibilità. A*, guidato da h_{com} , penalizza i nodi

Tabella 5: Confronto A* vs BFS – Scenario di comunicazione.

Metrica	A*	BFS
Criterio di esplorazione	$f(n) = g(n) + h_{\text{com}}(n)$	hop count
Penalità nodi senza visibilità	fattore 1,5 vs 0,8	nessuna
Vincolo 2 hop consecutivi ciechi	orientato dall'euristica	non gestito
Rischio di invalidazione	basso	medio
Nodi espansi (atteso)	$\ll V$	$\sim V$

senza segnale con un fattore 1,5 contro 0,8, orientando attivamente la ricerca verso sequenze di nodi visibili. La validazione post-ricerca rimane comunque necessaria per entrambi gli algoritmi, poiché né l'euristica né BFS garantiscono il rispetto del vincolo in senso stretto.

3.3 Un'osservazione generale

Il numero di nodi espansi è la metrica più significativa per confrontare l'efficienza dei due algoritmi. A* espande tipicamente molti meno nodi rispetto a BFS, perché scarta presto le direzioni poco promettenti. BFS invece espande tutti i nodi a distanza k prima di procedere al livello $k + 1$, indipendentemente da dove si trova il goal. Questa differenza diventa sempre più marcata al crescere della rete.

4 Conclusioni

Questo progetto mostra concretamente perché la scelta dell'algoritmo di ricerca non è mai neutra: A* e BFS trovano percorsi molto diversi sullo stesso grafo, e la differenza non è solo in termini di costo ma di quali vincoli riescono a rispettare.

A* si è dimostrato più adatto agli scenari con vincoli ambientali continui (radiazione, energia solare), dove la qualità dell'euristica guida la ricerca in modo diretto. Il pattern Strategy adottato per le euristiche si è rivelato la scelta giusta: permette di estendere il sistema a nuovi scenari semplicemente aggiungendo una nuova classe, senza toccare il codice degli algoritmi.

BFS rimane uno strumento valido come baseline e per scenari dove la topologia della rete è tale che il percorso più corto è anche quello più sicuro. La sua semplicità computazionale e l'assenza di parametri da tarare lo rendono una scelta affidabile in contesti meno vincolati.

Un'evoluzione naturale del sistema potrebbe prevedere l'integrazione di **Dijkstra** come terzo termine di confronto (ottimale per pesi, senza euristica), o approcci basati su **Reinforcement Learning** per apprendere automaticamente una policy di navigazione adattiva in ambienti dinamici – un'estensione particolarmente interessante nel contesto di missioni reali su superfici planetarie.

5 Immagini

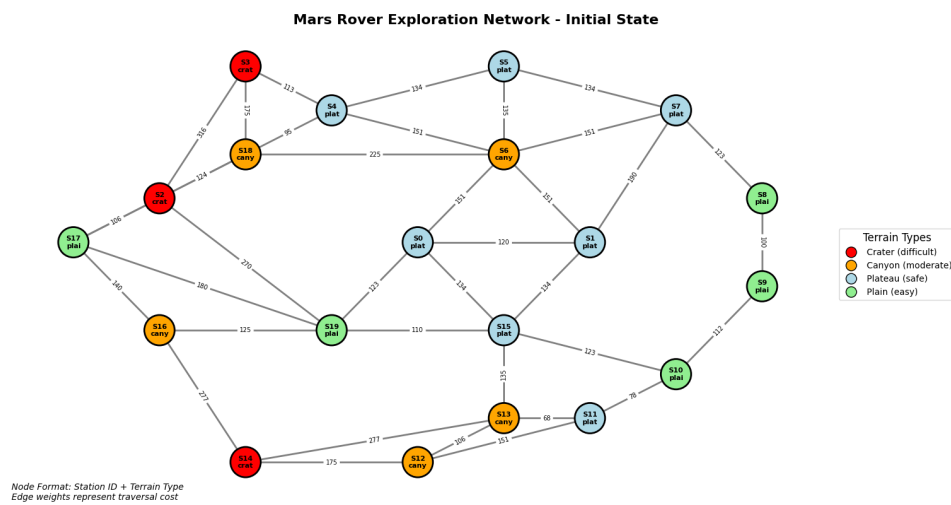


Figura 1: Rappresentazione grafica della rete di stazioni di esplorazione su Marte - scenario scientifico

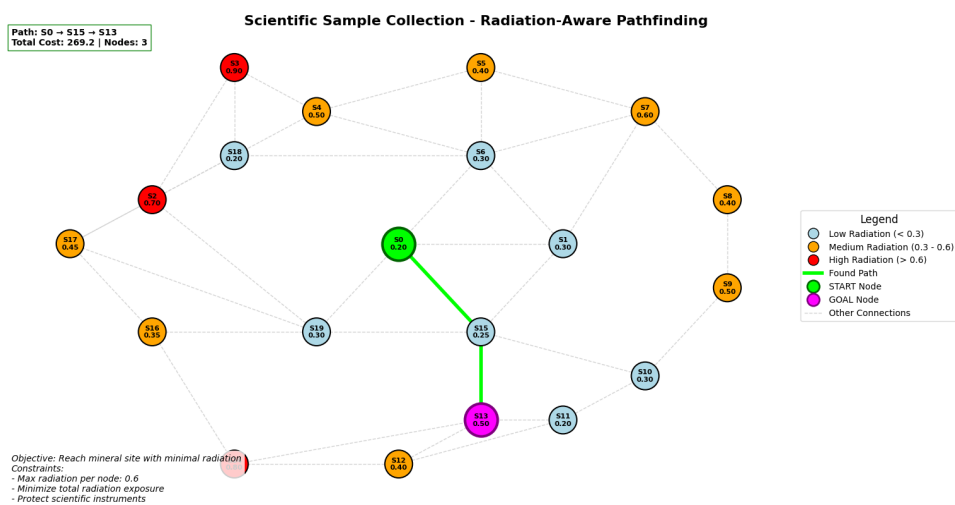


Figura 2: Percorso trovato con algoritmo A* e BFS - scenario scientifico

```

i 🚀 Initializing Mars Rover Exploration Network...
i 📡 Running scientific scenario: Scientific Sample Collection - Minimize radiation exposure
i 📍 Start: Station 0 (plateau)
i 🎯 Goal: Station 13 (canyon)
i 🧠 Algorithm: ASTAR
i ✅ Path found! S0 → S15 → S13
i 📊 Total cost: 269.16
i 📏 Path length: 3 nodes
  
```

Figura 3: Recap terminale - scenario scientifico

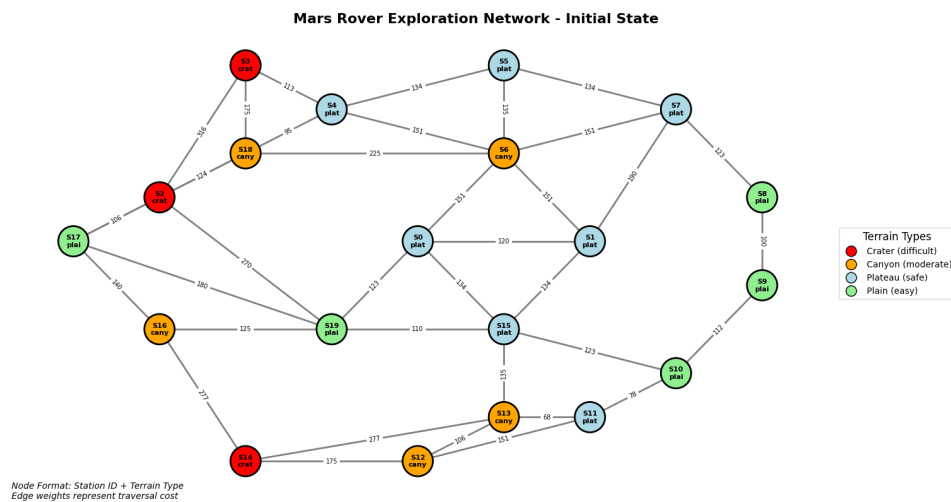


Figura 4: Rappresentazione grafica della rete di stazioni di esplorazione su Marte - scenario solare

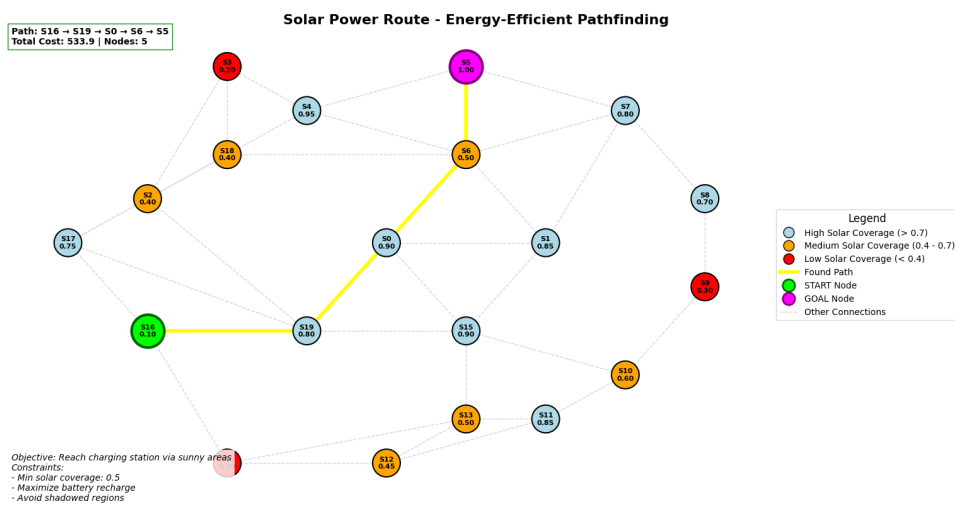


Figura 5: Percorso trovato con algoritmo A* e BFS - scenario solare

```

i Initializing Mars Rover Exploration Network...
i Running solar scenario: Solar Power Route - Maximize solar exposure for battery charging
i Start: Station 16 (canyon)
i Goal: Station 5 (plateau)
i Algorithm: ASTAR
i Path found! S16 → S19 → S0 → S6 → S5
i Total cost: 533.92
i Path length: 5 nodes

```

Figura 6: Recap terminale - scenario solare

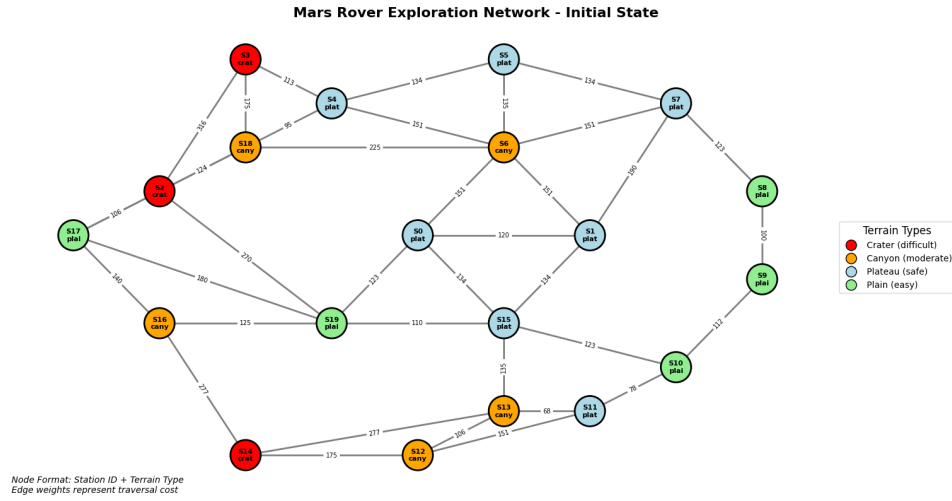


Figura 7: Rappresentazione grafica della rete di stazioni di esplorazione su Marte - scenario comunicazione

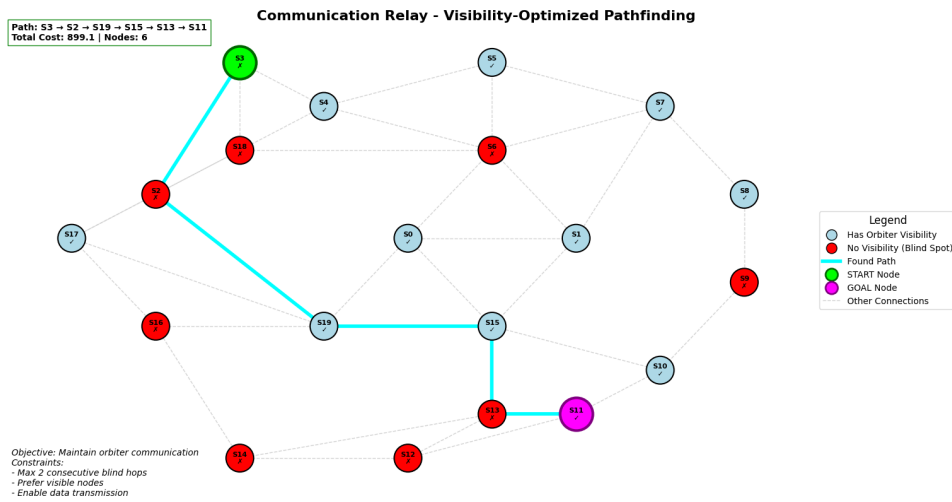


Figura 8: Percorso trovato con algoritmo BFS - scenario comunicazione

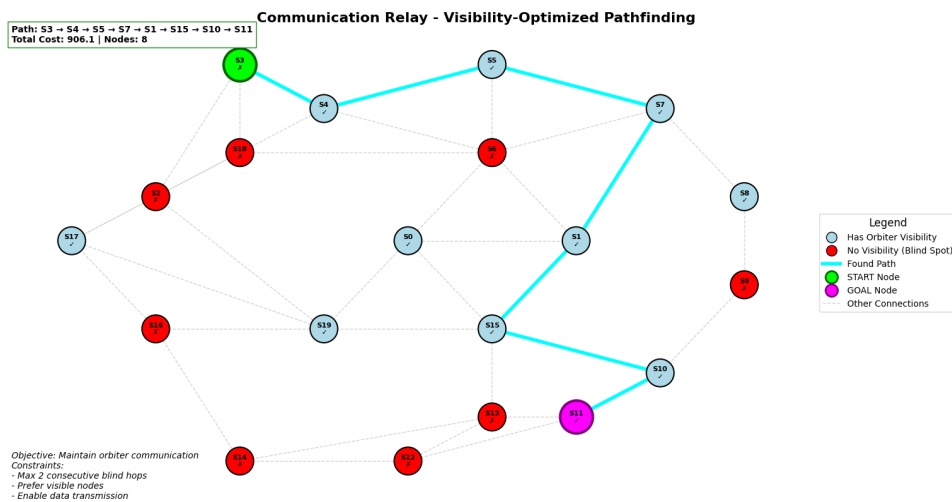


Figura 9: Percorso trovato con algoritmo A* - scenario comunicazione

```
i 🚀 Initializing Mars Rover Exploration Network...  
i 📡 Running communication scenario: Communication Relay - Maintain orbiter visibility  
i 📍 Start: Station 3 (crater)  
i 🎯 Goal: Station 11 (plateau)  
i 🧭 Algorithm: BFS  
i ✅ Path found! S3 → S2 → S19 → S15 → S13 → S11  
i 📊 Total cost: 899.14  
i 📏 Path length: 6 nodes
```

Figura 10: Recap terminale bfs - scenario comunicazione

```
i 🚀 Initializing Mars Rover Exploration Network...  
i 📡 Running communication scenario: Communication Relay - Maintain orbiter visibility  
i 📍 Start: Station 3 (crater)  
i 🎯 Goal: Station 11 (plateau)  
i 🧭 Algorithm: ASTAR  
i ✅ Path found! S3 → S4 → S5 → S7 → S1 → S15 → S10 → S11  
i 📊 Total cost: 906.13  
i 📏 Path length: 8 nodes
```

Figura 11: Recap terminale a* - scenario comunicazione